

Teste Técnico — Desenvolvedor Mobile (Flutter + Django)

Apresentação

Este teste tem como objetivo avaliar a capacidade do candidato de tomar decisões técnicas sólidas, estruturar código de forma organizada e aplicar os conceitos fundamentais das ferramentas propostas.

O Desafio

Desenvolver um sistema de **Gestão de Tarefas**, composto por uma API backend em **Django REST Framework** e um aplicativo mobile em **Flutter**.

O sistema deve permitir que usuários autenticados criem, visualizem, atualizem e removam tarefas, com suporte a atribuição de responsável, prioridade e histórico de alterações.

Backend — Django REST Framework

Autenticação

- Implementar autenticação via **JWT** utilizando `django-rest-framework-simplejwt`
- Expor os endpoints padrão de login, refresh e dados do usuário autenticado

Modelo de Tarefa

O modelo central da aplicação deve conter os seguintes campos:

Campo	Tipo	Observações
<code>id</code>	UUID	Gerado automaticamente
<code>titulo</code>	string	Obrigatório
<code>descricao</code>	text	Opcional
<code>status</code>	choice	<code>backlog</code> , <code>em_andamento</code> , <code>concluido</code>
<code>prioridade</code>	choice	<code>baixa</code> , <code>media</code> , <code>alta</code>
<code>criado_por</code>	FK → User	Preenchido automaticamente com o usuário autenticado

Campo	Tipo	Observações
atribuido_para	FK → User	Opcional, qualquer usuário cadastrado
data_limite	datetime	Opcional
criado_em	datetime	Automático
atualizado_em	datetime	Automático

Endpoints

Método	Endpoint	Descrição
POST	/api/auth/login/	Autenticação
POST	/api/auth/refresh/	Renovação do token
GET	/api/auth/me/	Dados do usuário autenticado
GET	/api/tarefas/	Listar tarefas com filtros e paginação
POST	/api/tarefas/	Criar tarefa
GET	/api/tarefas/{id}/	Detalhar tarefa
PATCH	/api/tarefas/{id}/	Atualizar tarefa parcialmente
DELETE	/api/tarefas/{id}/	Remover tarefa
GET	/api/tarefas/{id}/historico/	Listar histórico de alterações da tarefa

Regras de negócio

- Somente o criador da tarefa pode editá-la ou removê-la
- O campo `criado_por` não pode ser alterado via API
- A transição de `concluido` de volta para qualquer outro status não é permitida
- Qualquer usuário autenticado pode visualizar todas as tarefas

Histórico de alterações

Toda alteração em uma tarefa deve ser registrada automaticamente, contendo: campo alterado, valor anterior, valor novo, usuário responsável pela alteração e data/hora. A implementação dessa lógica faz parte da avaliação — a abordagem escolhida (signals, `save()` override, mixin de serializer, etc.) deve ser justificada no README.

Filtragem e paginação

O endpoint de listagem deve suportar:

- Filtro por `status` e `prioridade`
- Busca textual por `titulo`
- Ordenação por `criado_em` e `data_limite`
- Paginação com controle de `page` e `page_size`

Infraestrutura

- Banco de dados: **SQLite** (arquivo persistido via volume Docker)
- O backend deve ser inteiramente containerizado
- As migrations devem ser executadas automaticamente ao iniciar o container
- O projeto deve subir com o comando:

bash

```
docker compose up --build
```

Mobile — Flutter

Arquitetura

A aplicação deve seguir obrigatoriamente:

- **BLoC** para gerenciamento de estado (`flutter_bloc`)
- **Freezed** para modelagem de estados, eventos e entidades de domínio
- **JsonSerialization** (`json_serializable` + `build_runner`) para deserialização das respostas da API

A avaliação levará em conta a consistência da arquitetura adotada — como os eventos fluem, como os estados são modelados e como as camadas se comunicam.

Telas

Tela	Descrição
Login	Autenticação com validação de formulário
Lista de Tarefas	Listagem com filtro por status e paginação ou scroll infinito
Detalhe da Tarefa	Exibição completa dos dados, comentários e acesso ao histórico
Criar / Editar Tarefa	Formulário com validações; seleção de usuário para atribuição
Histórico	Apresentação cronológica das alterações registradas na tarefa

Requisitos técnicos

- Persistência do token JWT entre sessões (`flutter_secure_storage` ou equivalente)
 - Interceptor HTTP para injeção automática do header `Authorization` e renovação do token ao expirar
 - Tratamento explícito de erros da API com feedback visual adequado ao usuário
 - O código gerado pelo `build_runner` deve estar commitado no repositório
-

Entrega

- Repositório **público no GitHub** contendo o código-fonte do backend e do mobile
- O repositório deve refletir o desenvolvimento real — commits atômicos e descritivos são esperados
- Não serão aceitos repositórios com commit único

Estrutura mínima esperada

```
/
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── ...
├── mobile/
│   ├── pubspec.yaml
│   └── ...
├── docker-compose.yml
└── README.md
```

README

O README deve conter:

1. Instruções para subir o backend com Docker
 2. Instruções para rodar o app Flutter
 3. Variáveis de ambiente necessárias (com um `.env.example`)
 4. Decisões técnicas relevantes — especialmente a abordagem escolhida para o histórico de alterações e para a arquitetura BLoC no mobile
-

Critérios de Avaliação

Área	Peso
Autenticação JWT corretamente implementada (backend e mobile)	Alto
Lógica de histórico de alterações e sua justificativa	Alto
Consistência da arquitetura BLoC — eventos, estados, camadas	Alto
Aplicação correta de Freezed e JsonSerialization	Alto
Regras de negócio e permissões respeitadas no backend	Alto
Docker funcional com um único comando	Alto
Qualidade, legibilidade e organização do código	Médio
Tratamento de erros e experiência do usuário no app	Médio
README claro com justificativas técnicas	Médio
Histórico de commits organizado	Médio

Diferenciais

Os itens abaixo não são obrigatórios, mas serão considerados positivamente:

- Testes unitários no backend (`pytest-django`)
- Testes de BLoC no Flutter
- Documentação da API com Swagger (`drf-spectacular`)
- Soft delete nas tarefas
- Substituição do SQLite por PostgreSQL no Docker

Observações

- O uso de pacotes e bibliotecas externas é permitido e encorajado
 - O código deve ser de autoria própria
 - Dúvidas sobre o enunciado devem ser direcionadas ao recrutador responsável
-

O objetivo deste teste não é avaliar se você conhece todas as ferramentas listadas, mas como você raciocina, estrutura e justifica suas decisões técnicas.